

课程设计：电子时钟

毛宇航

2200017723

1 实验目的

以 Verilog 和 EGo 板设计电子时钟，且该时钟具有接近实际时间稳定计时，以按钮调整显示时间，秒表及其开始、暂停和复位，闹钟以及相应设置、清除、调整闹钟和闹钟提示等功能。

2 实验成果

<https://disk.pku.edu.cn/link/AA81330A2177114B88961D5A23316C5A05> 文件名：实验成果视频.mp4 有效期限：2024-09-16 13:08

3 源代码

3.1 clock.v

```
1 module led_clk_div(clk, rstn, div_clk);
2     input clk, rstn;
3     output div_clk;
4     reg[17:0] clk_reg;
5     assign div_clk = clk_reg[17];
6
7     always@(posedge clk)begin
```

```

8     if(!rstn)
9         clk_reg <= 18'b0;
10    else
11        clk_reg <= clk_reg+1;
12    end
13 endmodule
14
15 module led_display(clk, rstn, hour, minute, second, led_0, led_1, led_dn
    , alarmNotSet, mode);
16     input clk, rstn, alarmNotSet,mode;
17     input [6:0] hour, minute, second;
18     output reg[6:0] led_0, led_1;
19     output reg[5:0] led_dn;
20
21     always@(posedge clk or negedge rstn)
22         if(!rstn)
23             led_dn <= 6'b1;
24         else
25             led_dn <= {led_dn[4:0], led_dn[5]};
26
27     reg[3:0] cur_led0;
28     reg[3:0] cur_led1;
29
30     always@(*) begin
31         if(led_dn[0] == 1)
32             cur_led0 <= second%10;
33         else if(led_dn[1] == 1)
34             cur_led0 <= second/10;
35         else if(led_dn[2] == 1)
36             cur_led0 <= minute%10;
37         else if(led_dn[3] == 1)
38             cur_led0 <= minute/10;

```

```

39     else
40         cur_led0 <= 0;
41     end
42
43 always@(*)
44     if(led_dn[3] == 1)
45         cur_led1 <= minute/10;
46     else if(led_dn[4] ==1)
47         cur_led1 <= hour%10;
48     else
49         cur_led1 <= 0;
50
51 always@(*) begin
52     if(alarmNotSet ==1 && mode ==2'b10)
53         led_0 =7'b0000001;
54     else
55         case(cur_led0)
56             0: led_0 = 7'b1111110;
57             1: led_0 = 7'b0110000;
58             2: led_0 = 7'b1101101;
59             3: led_0 = 7'b1111001;
60             4: led_0 = 7'b0110011;
61             5: led_0 = 7'b1011011;
62             6: led_0 = 7'b1011111;
63             7: led_0 = 7'b1110000;
64             8: led_0 = 7'b1111111;
65             9: led_0 = 7'b1111011;
66             default: led_0 = 7'b0000000;
67         endcase
68     end
69 always@(*) begin
70     if(alarmNotSet ==1 && mode ==2'b10)

```

```

71         led_1 =7'b00000001;
72     else
73         case(cur_led1)
74             0: led_1 = 7'b11111110;
75             1: led_1 = 7'b01100000;
76             2: led_1 = 7'b1101101;
77             3: led_1 = 7'b1111001;
78             4: led_1 = 7'b0110011;
79             5: led_1 = 7'b1011011;
80             6: led_1 = 7'b1011111;
81             7: led_1 = 7'b1110000;
82             8: led_1 = 7'b1111111;
83             9: led_1 = 7'b1111011;
84             default: led_1 = 7'b00000000;
85         endcase
86     end
87
88 endmodule
89
90 module debounce(clk, rstn, button_in, button_out);
91     input clk, rstn, button_in;
92     output button_out;
93
94     reg button_out;
95     reg button_reg;
96     reg[15:0] counter;
97     // reg counter;
98
99     always @(posedge clk or negedge rstn)
100         if (!rstn)
101             counter <= 0;
102         else if (button_reg != button_in || |counter)

```

```

103         counter <= counter + 1;
104     else
105         counter <= 0;
106
107     always @(posedge clk or negedge rstn)
108         if (!rstn)
109             button_out <= 0;
110         else if (!counter)
111             button_out <= button_out;
112         else
113             button_out <= button_reg;
114
115     always @(posedge clk or negedge rstn)
116         if (!rstn)
117             button_reg <= 0;
118         else if (!counter)
119             button_reg <= button_reg;
120         else
121             button_reg <= button_in;
122
123     endmodule
124
125 module time_counter(clk, rstn, hour, minute, second, mode, change_time);
126     input clk, rstn;
127     input [1:0] mode;
128     input [5:0] change_time;
129     output[6:0] hour, minute, second;
130
131     reg [5:0] last_change_time;
132     //调时
133     integer k;
134     always @(posedge clk or negedge rstn)

```

```

135     if(!rstn) begin
136         k <=0;
137         last_change_time <= 6'b0;
138     end
139     else begin
140         last_change_time <= change_time;
141         if(change_time[0]==1 && last_change_time[0]==0 && mode==2'b01)
142             k <= k+1;
143         else if(change_time[1]==1 && last_change_time[1]==0 && mode==2
144             'b01)
145             k <= k-1;
146         else if(change_time[2]==1 && last_change_time[2]==0 && mode
147             ==2'b01)
148             k <= k+60;
149         else if(change_time[3]==1 && last_change_time[3]==0 && mode
150             ==2'b01)
151             k <= k-60;
152         else if(change_time[4]==1 && last_change_time[4]==0 && mode
153             ==2'b01)
154             k <= k+3600;
155         else if(change_time[5]==1 && last_change_time[5]==0 && mode
156             ==2'b01)
157             k <= (k-3600>=0)?(k-3600):0;
158     end
159
160 //计时
161 integer j;
162 always@(posedge clk or negedge rstn) begin
163     if(!rstn)
164         j <= 0;
165     else if(j >= 100000000)
166         j <= 0;

```

```

162         else if(mode != 2'b01) // 模式不为调时时计时
163             j <= j+1;
164     end
165     integer i;
166     always@(posedge clk or negedge rstn)
167         if(!rstn)
168             i <= 0;
169         else if(j==100000000)
170             i <= i+1;
171     integer ii;
172     always@(*) begin
173         ii = i+k;
174         //if(ii < 0) ii=0;
175     end
176     assign hour = ii/3600;
177     assign minute = (ii%3600)/60;
178     assign second = (ii%3600)%60;
179 endmodule
180
181 module stop_watch(clk, rstn, mode, pause, reset, hour, minute, second);
182     input clk, rstn, pause, reset;
183     input[1:0] mode;
184     output[6:0] hour, minute, second;
185
186     integer j;
187     always@(posedge clk or negedge rstn) begin
188         if(!rstn)
189             j <= 0;
190         else if(mode == 2'b11)begin
191             if(reset==1)
192                 j <= 0;
193             else if(j>=1000000)

```

```

194         j <= 0;
195     else if(pause != 0)
196         j <= j+1;
197     end
198     else
199         j <= 0;
200     end
201
202     integer i;
203     always@(posedge clk or negedge rstn) begin
204         if(!rstn)
205             i <= 0;
206         else if(mode ==2'b11)begin
207             if(reset==1)
208                 i <= 0;
209             else if(j==1000000)
210                 i <= i+1;
211         end
212     end
213
214     assign second = i%100;
215     assign hour = (i)/6000;
216     assign minute =(i%6000)/100;
217
218 endmodule
219
220 module alarm(clk, rstn, mode, change_time, resetAlarm, hour, minute,
221             second, alarmNotSet);
222     input clk, rstn, resetAlarm;
223     input[1:0] mode;
224     input[5:0] change_time;
225     output reg alarmNotSet ;

```



```

225     output [6:0] hour,minute, second;
226
227     reg [5:0] last_change_time;
228     integer k;
229     always@(posedge clk or negedge rstn)
230         if(!rstn) begin
231             k <= 0;
232             last_change_time <=0;
233             alarmNotSet <= 1;
234         end
235         else if(mode == 2'b10) begin
236             if(resetAlarm ==1) begin
237                 k <=0;
238                 last_change_time <= 6'b0;
239                 alarmNotSet <=1;
240             end
241             else begin
242                 last_change_time <= change_time;
243                 if(change_time[0]==1 && last_change_time[0] == 0 && mode
244                     == 2'b10) begin
245                     k <= k+1;
246                     alarmNotSet <= 0; end
247                 else if(change_time[1]==1 && last_change_time[1] == 0 &&
248                     mode == 2'b10) begin
249                     k <= (k-1>=0)?(k-1):0;
250                     alarmNotSet <= 0;end
251                 else if(change_time[2]==1 && last_change_time[2] == 0 &&
252                     mode == 2'b10) begin

```

```

253         k <= (k-60>=0)?(k-60):0;
254         alarmNotSet <= 0;end
255     else if(change_time[4]==1 && last_change_time[4] == 0 &&
mode == 2'b10) begin
256         k <= k+3600;
257         alarmNotSet <= 0;end
258     else if(change_time[5]==1 && last_change_time[5] == 0 &&
mode == 2'b10) begin
259         k <= (k-3600>=0)?(k-3600):0;
260         alarmNotSet <= 0;end
261     end
262 end
263 assign hour = k/3600;
264 assign minute = (k%3600)/60;
265 assign second = (k%3600)%60;
266 endmodule
267
268 module alarm_display(clk, rstn, hour_now, minute_now, second_now,
hour_set, minute_set, second_set, closeAlarm, mode, led);
269     input clk, rstn, closeAlarm;
270     input [1:0] mode;
271
272     input [6:0] hour_now, minute_now, second_now, hour_set, minute_set,
second_set; //now为当前, set为设定时间
273     output led;
274
275     assign led =(closeAlarm == 1)?0:(mode == 2'b00 && hour_now ==
hour_set && minute_now == minute_set && second_now == second_set)
?1:0;
276 endmodule
277
278 module top(clk ,rstn, led_0, led_1, led_dn, isCTmode, isAlarm,

```

```

isStopWatch, ct_button_in, pause, resetAlarm, alarm_led, closeAlarm);
279 input clk, rstn, isCTmode, isAlarm, isStopWatch,resetAlarm,
    closeAlarm;
280 input [5:0] ct_button_in;
281 input pause;
282 output[6:0] led_0;
283 output[6:0] led_1;
284 output[5:0] led_dn;
285 output alarm_led;
286
287 wire[1:0] mode;
288 assign mode =(isCTmode == 1)? 2'b01:(isAlarm == 1)?2'b10:(isStopWatch
    ==1)?2'b11:2'b00;
289
290 wire [6:0] hour_tc, minute_tc, second_tc;
291 wire led_clk;
292 led_clk_div lcd(clk, rstn, led_clk);
293 wire [5:0]ct_button_out;
294 debounce db0(clk, rstn, ct_button_in[0], ct_button_out[0]), db1(clk,
    rstn, ct_button_in[1], ct_button_out[1]), db2(clk, rstn,
    ct_button_in[2], ct_button_out[2]), db3(clk, rstn, ct_button_in
    [3], ct_button_out[3]), db4(clk, rstn, ct_button_in[4],
    ct_button_out[4]), db5(clk, rstn, ct_button_in[5], ct_button_out
    [5]);
295 time_counter tctr(clk, rstn, hour_tc, minute_tc, second_tc, mode,
    ct_button_out);
296
297 wire [6:0] hour_sw,minute_sw, second_sw;
298 stop_watch sw(clk, rstn, mode, pause, ct_button_out[4], hour_sw,
    minute_sw, second_sw);
299
300 wire [6:0] hour_am,minute_am,second_am;

```

```

301     wire alarmNotSet;
302     alarm am(clk, rstn, mode, ct_button_out, resetAlarm, hour_am,
              minute_am, second_am, alarmNotSet);
303     alarm_display ady(clk, rstn, hour_tc, minute_tc, second_tc, hour_am,
              minute_am, second_am, closeAlarm, mode, alarm_led);
304
305     wire [6:0] hour, minute, second;
306     assign hour = (mode == 2'b11)?hour_sw:(mode == 2'b10)?hour_am:hour_tc;
307     assign minute = (mode == 2'b11)?minute_sw:(mode == 2'b10)?minute_am:
              minute_tc;
308     assign second = (mode == 2'b11)?second_sw:(mode == 2'b10)?second_am:
              second_tc;
309     led_display ldy(led_clk, rstn, hour, minute, second, led_0, led_1,
              led_dn, alarmNotSet, mode);
310 endmodule

```

代码解释：

- 1) led_clk_div模块用以将 led 的 clk 分频，以使肉眼能观测到数码管同时亮起。
- 2) led_display模块用以控制 led 的多路选择和数字显示。
- 3) debounce模块用以将按键消抖。
- 4) time_counter模块用以实现时钟基础的计时和调时。
- 5) stop_watch模块用以实现秒表的所有功能。
- 6) alarm模块用以实现闹钟的调时和复位。
- 7) alarm_display模块用以实现闹钟计时到位后 LED 灯的亮起。
- 8) top模块中将上述模块依次实例化，并以 mode 多路选择控制 LED 灯显示的时间。

3.2 timecounter_tb.v

```

1 module timecounter_tb;
2     reg clk,rstn,isCTmode, isAlarm, isStopWatch, pause;
3     reg [5:0] ct_button_in;
4     wire[6:0] led_0, led_1;
5     wire [5:0] led_dn;
6     top topp(clk ,rstn, led_0, led_1, led_dn, isCTmode, isAlarm,
7         isStopWatch, ct_button_in, pause);
8
9     initial begin
10         isStopWatch = 0;
11         isCTmode = 1;
12         isAlarm = 0;
13         rstn =0;
14         clk =1;
15         pause =1;
16         ct_button_in =6'b0000000;
17
18         #5 rstn =1;
19
20         #100 ct_button_in =6'b1000000; #20 ct_button_in =6'b0000000;
21         #100 ct_button_in =6'b0100000; #20 ct_button_in =6'b0000000;
22         #100 ct_button_in =6'b0010000; #20 ct_button_in =6'b0000000;
23         #100 ct_button_in =6'b0001000; #20 ct_button_in =6'b0000000;
24         #100 ct_button_in =6'b0000100; #20 ct_button_in =6'b0000000;
25         #100 ct_button_in =6'b0000010; #20 ct_button_in =6'b0000000;
26         #100 ct_button_in =6'b0000001; #20 ct_button_in =6'b0000000;
27     end
28     always #5 clk=~clk;
29 endmodule

```

代码解释：timecounter_tb模块对其中的调时功能进行仿真以验证功能的正确实现。

3.3 stop_watch_tb.v

```
1 module stop_watch_tb;
2     reg clk,rstn,isCTmode, isAlarm, isStopWatch, pause;
3     reg [5:0] ct_button_in;
4     wire[6:0] led_0, led_1;
5     wire [5:0] led_dn;
6     top topp(clk ,rstn, led_0, led_1, led_dn, isCTmode, isAlarm, isStopWatch
7         , ct_button_in, pause);
8
9     initial begin
10         isStopWatch = 1;
11         isCTmode = 0;
12         isAlarm = 0;
13         rstn = 0;
14         clk = 0;
15         pause =1;
16         ct_button_in =6'b000000;
17
18         #5 rstn = 1;
19     end
20     always #5 clk=~clk;
21 endmodule
```

代码解释：stop_watch_tb模块对其中的秒表功能进行仿真以验证功能的正确实现。

4 实验结果

- 1) 将clock.v中debounce模块的寄存器count设为1位,led_display模块以分频前的clk实例化，再对timecounter_tb.v的仿真验证中，可得如下波形图

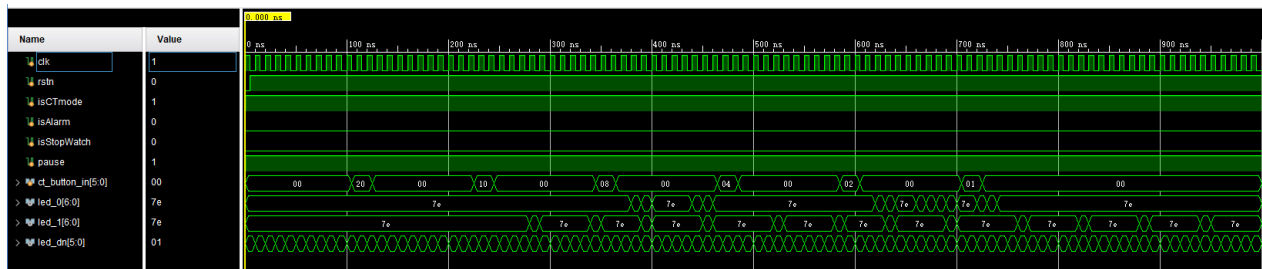


图 1: 调时功能仿真波形图

可观察到，led_0与led_1的值随着ct_button_in的值而变化，即按下按钮后，led 灯显示发生变化，功能实现。

2) 将上述值改回，再对stop_watch_tb.v的仿真验证，可得如下波形图

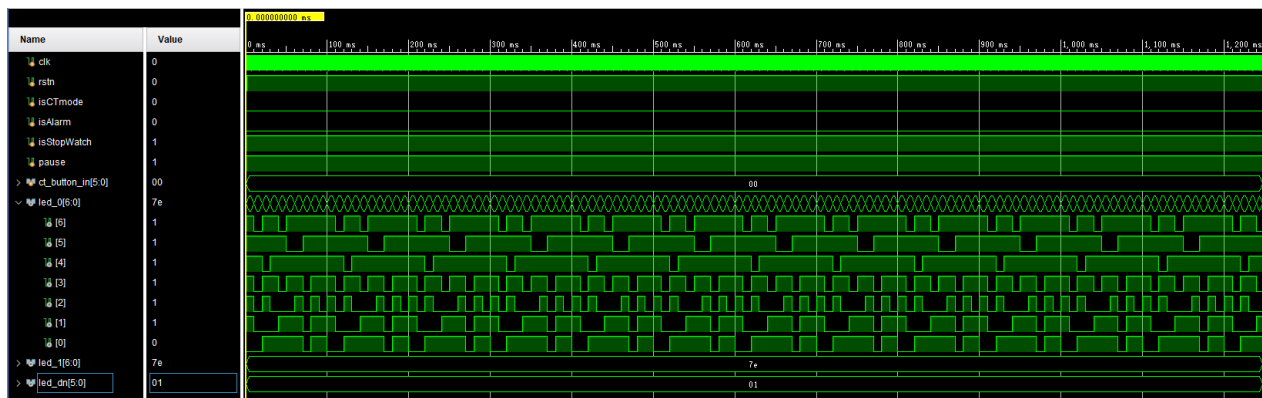


图 2: 秒表功能仿真波形图

可观察到，将仿真时间放在 ms 量级下时，led_0的值随着clk的变化而变化，秒表功能实现。

3) 逻辑综合后，可得电路图：

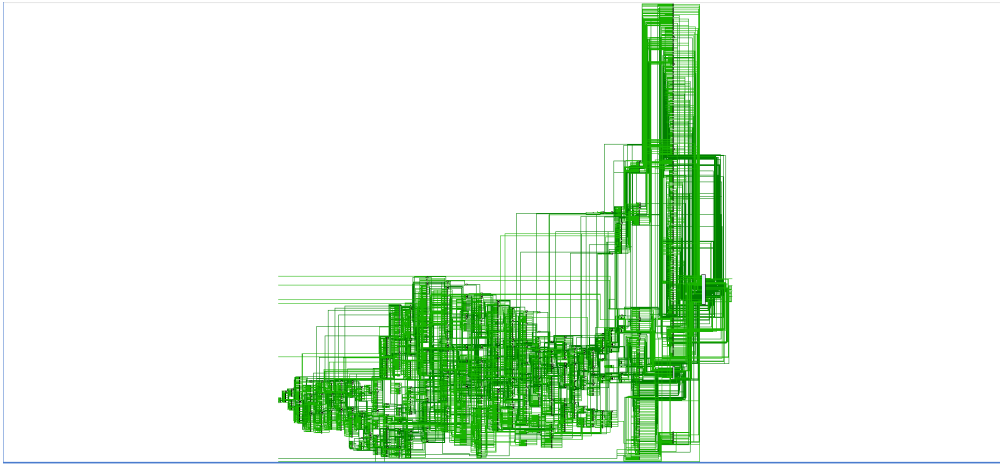


图 3: Synthesis-Schematic

4) 设置约束并布局布线后，可得 Device 图：

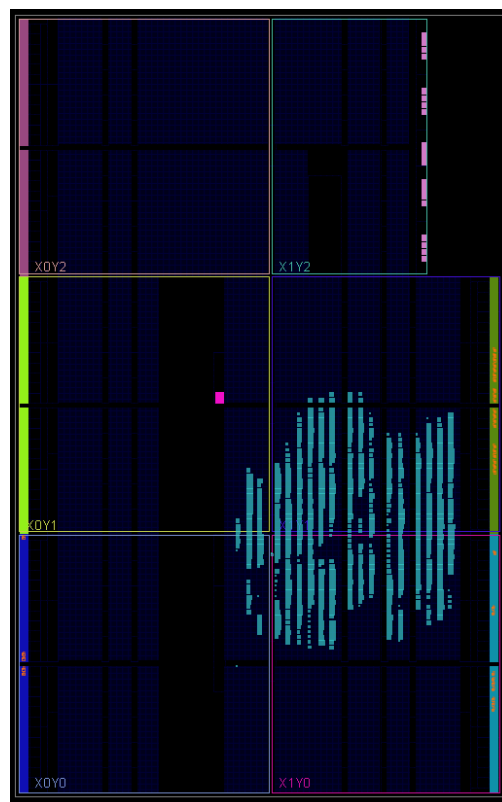


图 4: Implementation-Device

- 5) 生成比特流，将 EGo1 板与电脑连接，烧录至板后，可依次验证时钟计时、调时、秒表及其暂停复位、闹钟及其设置、清除、亮灯提醒和关闭功能。（详见“实验成果”中视频链接）